CV-Tools Filter Reference

Every filter, parameter, and caveat in one document

Filter Reference

Every filter is a plain function taking an image as its first argument and returning a new image. Registry names (the `name` column) are what appear in JSON presets and reports.

Registry name	Function	Module	Sprint
`clahe`	`apply_clahe`	`cv_tools.filters.clahe`	1
`contrast_brightness`	`adjust_contrast_brightne`	`cv_tools.filters.contrast_brightness`	1
`auto_contrast`	`auto_contrast`	`cv_tools.filters.contrast_brightness`	1
`levels`	`adjust_levels`	`cv_tools.filters.levels`	1
`auto_levels`	`auto_levels`	`cv_tools.filters.levels`	1
`histeq`	`histogram_equalization`	`cv_tools.filters.histogram_equalization`	1
`roi_crop`	`roi_crop`	`cv_tools.filters.roi`	1
`roi_draw`	`roi_draw`	`cv_tools.filters.roi`	1
`crop`	`crop`	`cv_tools.filters.crop_resize`	1
`resize`	`resize`	`cv_tools.filters.crop_resize`	1
`rotate`	`rotate`	`cv_tools.filters.crop_resize`	1
`flip`	`flip`	`cv_tools.filters.crop_resize`	1
`sharpen`	`unsharp_mask`	`cv_tools.filters.sharpen`	2
`sharpen_laplacian`	`laplacian_sharpen`	`cv_tools.filters.sharpen`	2
`gaussian_blur`	`gaussian_blur`	`cv_tools.filters.smoothing`	2
`median_filter`	`median_filter`	`cv_tools.filters.smoothing`	2
`bilateral_filter`	`bilateral_filter`	`cv_tools.filters.smoothing`	2
`canny`	`canny_edges`	`cv_tools.filters.edge_detection`	2
`auto_canny`	`auto_canny`	`cv_tools.filters.edge_detection`	2
`sobel`	`sobel_edges`	`cv_tools.filters.edge_detection`	2
`laplacian`	`laplacian_edges`	`cv_tools.filters.edge_detection`	2

ela	error_level_analysis	cv_tools.filters.ela	3
fft_spectrum	fft_magnitude_spectrum	cv_tools.filters.fft_analysis	3
fft_filter	fft_filter	cv_tools.filters.fft_analysis	3
remove_periodic	remove_periodic_noise	cv_tools.filters.fft_analysis	3
noise_map	noise_map	cv_tools.filters.noise_analysis	3
clone_detect	highlight_clones	cv_tools.filters.clone_detection	3
ghost	ghost_map	cv_tools.filters.jpeg_ghost	3
deblur_motion	deblur_motion	cv_tools.filters.motion_deblur	4
deblur_defocus	deblur_defocus	cv_tools.filters.motion_deblur	4
curves / s_curve	apply_curve / s_curve	cv_tools.filters.curves	—
white_balance / white_balance_patch / temperature	auto_white_balance / white_balance_from_patch / adjust_temperature	cv_tools.filters.white_balance	
saturation / vibrance / desaturate / selective_saturation	see module	cv_tools.filters.saturation	
color_balance / cmyk / channel_mixer	see module	cv_tools.filters.color_balance	
invert / invert_channel / invert_luminance / solarize	see module	cv_tools.filters.invert	—
nl_means / nl_means_auto	nl_means_denoise	cv_tools.filters.nl_means_denoise	
upscale	upscale	cv_tools.filters.super_resolution	
local_contrast / detail_enhance / multiscale_detail / texture_boost	see module	cv_tools.filters.detail_enhancement	
perspective / auto_perspective	correct_perspective	cv_tools.filters.perspective_correction	

`barrel` / `fisheye`	`correct_barrel_distortion` / `correct_fisheye`	`cv_tools.filters.fisheye_correction`
`pixel_aspect` / `fit_aspect`	`correct_pixel_aspect` / `fit_to_aspect`	`cv_tools.filters.aspect_ratio`
`undistort`	`undistort_with_file`	`cv_tools.filters.undistort`
`blocking_map` / `deblock`	see module	`cv_tools.filters.compression_analysis`
`stain`	`extract_stain`	`cv_tools.filters.color_deconvolution`
`component` / `bit_plane`	`extract_component` / `extract_bit_plane`	`cv_tools.filters.component_separation`
`redact`	`redact_region`	`cv_tools.filters.redaction`

Sprint 1 — Adjust & Correct

CLAHE — `clahe`

Adaptive contrast enhancement applied to tiles, with contrast limiting to avoid amplifying noise. The workhorse for dark or low-contrast CCTV footage.

Param	Type	Default	Notes
`clip_limit`	float	`2.0`	Higher = more local contrast, more noise
`tile_grid_size`	int or (rows, cols)	`8`	`8` means 8×8 tiles
`color_mode`	str	`'lab'`	`lab`, `hsv`, `yuv`, `channelwise`, `luminance`

`lab`, `hsv` and `yuv` equalize a single luminance channel and leave chroma alone, so colors stay stable. `channelwise` equalizes R, G and B independently and **will** shift color — use it only when you want that.

CLI: `--clahe clip=3.0 tile=8x8 mode=lab`

`apply_clahe_grid(image, clip_limits, tile_grid_sizes)` renders a labelled grid of parameter combinations for picking values quickly.

Contrast & Brightness — `contrast_brightness`

`output = (input - 128) * contrast + 128 + brightness`, then gamma.

Param	Type	Default	Notes
`brightness`	float	`0.0`	Offset, -255 to 255
`contrast`	float	`1.0`	1.0 = unchanged
`gamma`	float	`1.0`	<1 darker midtones, >1 brighter
`channel`	str or None	`None`	`r`, `g`, `b` to target one channel

CLI: `--brightness 30`, `--contrast 1.5`, `--gamma 0.8` (each is a separate chain step)

Auto Contrast — `auto_contrast`

Stretches the luminance histogram to the full range.

Param	Type	Default	Notes
`cutoff`	float	`0.0`	Percent of darkest/brightest pixels to ignore (0–50)

CLI: `--auto-contrast` or `--auto-contrast 2`

Levels — `levels`

Maps input range `[black_point, white_point]` onto `[output_black, output_white]` with a gamma curve on the midtones.

Param	Type	Default
`black_point`	float	`0`
`gamma`	float	`1.0`
`white_point`	float	`255`
`output_black`	float	`0`
`output_white`	float	`255`
`channel`	str or None	`None`

Raises `ValueError` if `black_point >= white_point`.

CLI: `--levels 20,1.0,220` (black, gamma, white)

Auto Levels — `auto_levels`

Param	Type	Default	Notes
`per_channel`	bool	`False`	`True` stretches R/G/B independently and can shift color

CLI: `--auto-levels`

Histogram Equalization — `histeq`

Global equalization — flattens the whole histogram at once. Stronger and blunter than CLAHE; prone to amplifying noise in flat regions.

Param	Type	Default	Notes

`color_mode`	str	`'lab'`	`lab`, `hsv`, `yuv`, `channelwise`, `grayscale`
`mask`	ndarray or None	`None`	Restrict the region used to build the histogram

CLI: `--histeq mode=lab`

ROI Crop — `roi_crop`

Crops to a region, **clipped** to image bounds — an oversized region silently shrinks.

Param	Type
`x`, `y`, `width`, `height`	int

CLI: `--roi 100,100,300,200`

ROI Draw — `roi_draw`

Draws a rectangle to mark a region without altering pixels underneath (unless `filled`).

Param	Type	Default
`x`, `y`, `width`, `height`	int	—
`color`	(r, g, b)	`(255, 0, 0)`
`thickness`	int	`2`
`label`	str or None	`None`
`filled`	bool	`False`
`alpha`	float	`0.3`

CLI: `--draw-roi 265,295,120,46`

Crop — `crop`

Same geometry as `roi_crop`, but **raises** `ValueError` when the region falls entirely outside the image instead of returning a clipped result. Use it when an out-of-bounds crop should be an error.

CLI: `--crop 100,100,300,200`

Resize — `resize`

Param	Type	Notes
`width`	int or None	Alone, height follows aspect ratio
`height`	int or None	Alone, width follows aspect ratio
`scale`	float or None	Used when width/height are absent
`interpolation`	str	`auto`, `nearest`, `bilinear`, `bicubic`, `lanczos`, `area`

`auto` picks `INTER_AREA` for downscaling and `INTER_LANCZOS4` for upscaling. For forensic work prefer `nearest` when you need to inspect pixels without resampling.

CLI: `--resize 800x600`, `--resize 800x`, `--resize x600`, `--resize 50%`, `--resize 0.5`, combined with `--interpolation lanczos`.

Rotate — `rotate`

Expands the canvas so no content is cropped.

Param	Type	Default
`angle`	float	— (degrees, counter-clockwise)
`center`	(x, y) or None	`None` = image center
`scale`	float	`1.0`
`border_mode`	str	`'constant'` (`replicate`, `reflect`, `wrap`)
`border_value`	(r, g, b)	`(0, 0, 0)`

CLI: `--rotate 90`

Flip — `flip`

Param	Type	Values
`direction`	str	`horizontal`, `vertical`, `both`

CLI: `--flip horizontal`

Sprint 2 — Enhance

Unsharp Mask — `sharpen`

Subtracts a blurred copy to isolate detail, then adds it back scaled by `amount`.

Param	Type	Default	Notes
`amount`	float	`1.0`	0 = no change; above 2 usually looks artificial
`radius`	float	`1.0`	Blur sigma. Small = fine detail, large = local contrast.
`threshold`	int	`0`	Minimum local contrast (0–255) before a pixel is sharpened

`threshold` is the noise control: raise it and smooth areas — where noise lives — are left alone while genuine edges still get sharpened. Sharpen **after** denoising, never before.

CLI: `--sharpen amount=1.5 radius=1.0 threshold=4`

`sharpen_grid(image, amounts, radii)` renders a labelled parameter grid, like `apply_clahe_grid`.

Laplacian Sharpen — `sharpen_laplacian`

`output = input - strength × laplacian(input)`. Harsher and more noise-sensitive than an unsharp mask, but needs no radius choice.

Param	Type	Default
`strength`	float	`1.0`
`kernel_size`	int	`3` (odd, 1–31)

CLI: `--sharpen-laplacian strength=1.0 kernel=3`

Gaussian Blur — `gaussian_blur`

Param	Type	Default	Notes
`radius`	float	`2.0`	Gaussian sigma in pixels

`kernel_size`	int	`0`	0 derives the kernel from the radius — normally what you want

General-purpose smoothing. Blurs edges along with noise; prefer bilateral when edges matter.

CLI: `--gaussian 1.5` (bare `--gaussian` uses 2.0)

Median Filter — `median_filter`

Param	Type	Default	Notes
`kernel_size`	int	`3`	Odd, 3 or greater

The standard remedy for salt-and-pepper / impulse noise. Output only ever contains values already present nearby, so plateaus and edges survive intact where a blur would smear them.

CLI: `--median 3` (bare `--median` uses 3)

Bilateral Filter — `bilateral_filter`

Param	Type	Default	Notes
`diameter`	int	`9`	Neighbourhood diameter; 0 derives it from `sigma_space`
`sigma_color`	float	`75.0`	Color-difference tolerance
`sigma_space`	float	`75.0`	Spatial extent

Weights neighbours by both distance and color similarity, so it smooths sensor noise inside regions without bleeding across boundaries. The slowest of the three smoothing filters.

CLI: `--bilateral d=9 color=75 space=75`

Sprint 2 — Analyze

All four edge detectors return a **single-channel** uint8 map, so they convert a color image to grayscale mid-chain.

Canny — `canny`

Param	Type	Default	Notes
`low_threshold`	float	`100`	Weak edges survive only if connected to a strong one
`high_threshold`	float	`200`	A 1:2 or 1:3 low:high ratio is the usual starting point
`aperture_size`	int	`3`	3, 5, or 7
`l2_gradient`	bool	`False`	Exact L2 magnitude instead of the cheaper L1
`blur_sigma`	float	`0.0`	Gaussian pre-blur — noisy footage needs one

Output is binary (0 or 255).

CLI: `--canny 50,150`, with `--blur-first 1.5` to set the pre-blur.

Auto Canny — `auto_canny`

Derives both thresholds from the image's median intensity, which suits batches of frames whose exposure varies. Falls back to 50/150 when the median collapses both thresholds onto the same value (a near-black or near-white frame).

Param	Type	Default	Notes
`sigma`	float	`0.33`	Spread around the median, 0–1. Larger keeps more edges.
`blur_sigma`	float	`0.0`	Gaussian pre-blur

CLI: `--auto-canny` or `--auto-canny 0.4`

Sobel — `sobel`

Param	Type	Default	Notes

`dx`	int	`1`	Horizontal derivative order (0 or 1)
`dy`	int	`1`	Vertical derivative order (0 or 1)
`kernel_size`	int	`3`	Odd, 1–7
`normalize`	bool	`True`	Stretch the result to fill 0–255

With both `dx` and `dy` set you get the gradient magnitude; with one you get that single directional derivative. Continuous-valued, unlike Canny's binary output.

CLI: `--sobel dx=1 dy=1 kernel=3`

Laplacian — `laplacian`

Param	Type	Default
`kernel_size`	int	`3` (odd, 1–31)
`normalize`	bool	`True`
`blur_sigma`	float	`0.0`

Responds to intensity change in all directions at once — and to noise just as eagerly, so a small `blur_sigma` is usually worth it.

CLI: `--laplacian kernel=3 blur=1.0`

Histogram (not a chain step)

`histogram.py` analyzes rather than transforms, so it is exposed as CLI output options instead of registry filters.

Function	Returns
`compute_histogram(image, bins=256, normalize=False)`	Dict of channel name → bin counts
`histogram_stats(image)`	Per-channel mean, median, std, min, max, p1, p99, clipping %
`dynamic_range_used(image)`	Fraction of 0–255 spanned between p1 and p99
`render_histogram(image, ...)`	RGB chart image

Clipping percentages are the forensically important part: pixels stuck at 0 or 255 have lost their original values, and no enhancement recovers them. A low `dynamic_range_used` means levels or CLAHE still has room to work.

`render_histogram` accepts `width`, `height`, `bins`, `log_scale` (reveals sparse tails a linear plot flattens to nothing), `show_grid`, `background`, and `channels` to restrict which curves are drawn.

CLI: `--histogram chart.png`, `--histogram-log`, `--hist-stats`

`edge_density(edges, threshold=0)` gives the fraction of pixels carrying an edge. It compares focus across frames of the same scene, but it is not a general sharpness measure — blurring an image whose only feature is one strong edge spreads that edge over more pixels and raises the density.

Sprint 3 — Forensic

Read this before using any of them. These filters find things *worth examining*. None of them establishes that an image was manipulated, and each has a failure mode that produces convincing-looking evidence for a false conclusion. The caveats below are part of the tool, not disclaimers around it.

Error Level Analysis — `ela`

Re-compresses the image as JPEG and amplifies the difference. A region with a different compression history than its surroundings may show a different error level.

Param	Type	Default	Notes
`quality`	int	`90`	Re-compression quality, 1–100. Aim near the original's.
`scale`	float	`0`	Brightness multiplier; 0 auto-scales so the peak error hits 255
`grayscale`	bool	`False`	Collapse per-channel error into one channel

Limits. Only meaningful on a JPEG original — re-saving as PNG, or a second full-image JPEG save, erases the signal completely. Bright areas track edge density and texture as much as editing history, so busy regions always look hot. A clean map does not mean the image is authentic.

CLI: `--ela quality=90 gray=true`, `--ela-stats [QUALITY]`

`ela_stats(image, quality, block_size)` returns the mean/max error, a per-block mean grid, and the hottest block with a z-score saying how far it stands above the rest. `recompress(image, quality)` exposes the JPEG round-trip on its own.

FFT Magnitude Spectrum — `fft_spectrum`

Param	Type	Default	Notes
`log_scale`	bool	`True`	Without it the DC term dwarfs everything and the plot is one dot
`normalize`	bool	`True`	Stretch to fill 0–255

DC is at the centre. Periodic structure — interlacing, halftone screens, scanner banding — appears as discrete bright points away from that centre.

CLI: `--fft log=true`

Frequency Filter — `fft_filter`

Param	Type	Default	Notes
`filter_type`	str	`'lowpass'`	`lowpass`, `highpass`, `bandpass`
`cutoff`	float	`30.0`	Radius in pixels from DC
`cutoff_high`	float	`0.0`	Upper radius, bandpass only
`soft`	bool	`True`	Gaussian-edged mask. A hard edge causes ringing that looks like real image content.

Returns single-channel output.

CLI: `--fft-filter type=highpass cutoff=20`

Periodic Noise Removal — `remove_periodic`

Finds isolated spectral peaks and notches them out, removing a repeating pattern with far less damage than a blur would do. The mirrored peak is notched too, since a real image's spectrum is symmetric about DC.

Param	Type	Default	Notes
`peaks`	list or None	`None`	Detected automatically when omitted
`notch_radius`	float	`4.0`	Radius of the Gaussian notch on each peak
`min_radius`	float	`10.0`	Ignore this radius around DC — low frequencies are the image itself
`threshold`	float	`4.0`	Standard deviations above the local background for a peak to count

Some residual pattern usually survives at the image borders: the FFT treats the image as tiling the plane, and the discontinuity at the edges spreads energy across the spectrum.

CLI: `--remove-periodic notch=4`

`detect_periodic_peaks(image, ...)` returns the peaks alone, so you can inspect them before notching.

Noise Map — `noise_map`

Param	Type	Default	Notes
`block_size`	int	`32`	Smaller localises better but estimates each block less reliably
`normalize`	bool	`True`	Stretch to fill 0–255
`upscale`	bool	`True`	Resize the block grid back to the input's dimensions

Bright means noisier. Sensor noise should be fairly even across an untouched frame, so a region that differs markedly came from somewhere else — a different camera, a different resize, or a denoise pass applied to that region alone. Heavy texture also raises the reading.

CLI: `--noise-map 32`, `--noise-stats`

`estimate_noise(image)` returns the global sigma via Immerkaer's method; `estimate_snr` gives dB (using the image's own std as signal, so a flat image scores low however clean); `noise_report` adds per-block statistics and a `uniformity` ratio.

Clone Detection — `clone_detect`

Finds regions duplicated from elsewhere in the same image. Blocks are described by their low-frequency DCT coefficients, sorted so near-identical blocks become neighbours, and a shift shared by many pairs is reported as a duplicated region.

Param	Type	Default	Notes
`step`	int	`1`	**Only shifts that are multiples of this are detectable**
`block_size`	int	`16`	Side length of each compared block
`coefficients`	int	`4`	Size of the top-left DCT square kept as the descriptor
`quantization`	float	`4.0`	Descriptor rounding; larger tolerates more compression, matches more falsely
`min_distance`	float	`0`	Minimum separation for a pair to count; 0 uses 2×`block_size`
`min_matches`	int	`8`	Pairs needed before a shift is reported

`min_variance`	float	`12.0`	Featureless blocks below this are skipped
`max_blocks`	int	`300000`	Memory guard; raises rather than allocating gigabytes

The `step` constraint is the one that catches people out. Blocks are sampled on a grid of that stride, so a region moved by 190 pixels is invisible to a stride of 8 — the copy is sampled at a different phase from the original and the descriptors do not match. The default of 1 is exhaustive. Raising it is a quick screening pass, not a search.

On a large image, crop to a region with `--roi` rather than raising `step`.

Limits. Genuine repetition — brick walls, windows, tiled floors, text — is duplication, and will be reported. This locates duplication, not intent.

CLI: `--clone-detect block=16 step=1 matches=8 variance=12`, `--clone-stats`

`detect_copy_move` returns the full result dict (mask, shift vectors, block counts); `draw_clone_regions` tints it onto the image.

JPEG Ghost Detection — `ghost`

Recompresses the image across a range of JPEG qualities and diffs each pass against the source, the same trick ELA uses once. Re-quantising an already-JPEG'd region at its own prior quality is nearly lossless, so each block's error-versus-quality curve dips sharply right at that quality — the "ghost". A block's minimum locates its likely prior compression quality from pixel evidence alone, even once the file's own quantisation tables are gone.

Param	Type	Default	Notes
`qualities`	list[int]	`50,55,...,100`	Ascending quality steps to sweep
`block_size`	int	`16`	Side length of the analysis blocks
`upscale`	bool	`True`	Resize the block grid back to the input's dimensions

The output map encodes, per block, the *index* into `qualities` of the best match — darker means an earlier (lower-quality) step. A region whose shade differs sharply from its surroundings had a different JPEG history.

Limits. A uniform JPEG resave of the whole composite is a blind spot: every block then shares one true last quality, and its trivially-near-zero dip there swamps any subtler trace of what a region was compressed at before a splice. The technique reads a composite that was never unified by a later full-frame JPEG save — a PNG built from JPEG sources is the common case it catches. Flat, low-texture regions dip only shallowly at every quality and read as ambiguous by design.

CLI: `--ghost block=16 min=50 max=100 step=5`, `--ghost-stats`

`ghost_map` returns the visual map; `ghost_report` adds the dominant quality and the list of outlier blocks.

Metadata Forensics (not a chain step)

Reads the container rather than the pixels: EXIF tags, JPEG application segments, and whether what the metadata claims matches what the image actually is. `metadata_report(path)` takes a file path, not an image, so it is a stats flag rather than a filter.

Check	Severity	What it means
`editing_software`	flag	`Software` names a known editor (matched against `EDITOR_SIGNATURES`, so camera firmware strings do not trigger it)
`modified_after_capture`	flag	`DateTime` is later than `DateTimeOriginal` — written again after the shutter fired
`timestamp_disorder`	flag	`DateTimeDigitized` precedes `DateTimeOriginal`, which capture order forbids
`dimension_mismatch`	flag	EXIF's recorded dimensions disagree with the actual ones — resized or cropped since capture
`photoshop_segment`	flag	An APP13 Photoshop resource block is embedded
`no_exif`	info	A format that normally carries EXIF has none
`no_camera_identification`	info	EXIF present but no `Make` or `Model`
`xmp_segment`	info	An XMP packet is embedded, which often records an editing history the EXIF does not

This is the cheapest check available and the easiest to defeat. Metadata is plain text in a header: anyone can edit or strip it, and most messaging and social platforms strip it wholesale on upload. So a clean header proves nothing — it is the normal state of a file that has been through WhatsApp — and an editor's name proves nothing either, since cropping, rotating and format conversion all leave one.

The contradictions are the part worth attention. A tag that disagrees with the pixels, or with another tag, is harder to produce by accident than a suspicious-looking name is.

CLI: `--metadata-stats`

`read_exif` returns the tags as a plain dict; `detect_editing_software` and `check_timestamps` are the individual checks, usable on an EXIF dict you already hold.

Wiener Deblurring — `deblur_motion`, `deblur_defocus`

Inverts a known blur. Naive inversion divides by the blur's frequency response, which is near zero at some frequencies, so noise there is amplified without limit. The Wiener filter tempers this: `F = G · conj(H) / (|H|² +`

`K)`, where `K` is `noise_power`.

Param	Type	Default	Notes
`length`	float	`15.0`	Motion extent in pixels (`deblur_motion`)
`angle`	float	`0.0`	Motion direction in degrees, 0 = horizontal (`deblur_motion`)
`radius`	float	`5.0`	Defocus circle radius (`deblur_defocus`)
`noise_power`	float	`0.01`	Raise on noisy footage to trade sharpness for stability

The input is reflect-padded before the transform and cropped afterwards, which keeps the FFT's wraparound — where the left edge convolves with the right — out of the result.

Limits. You must supply the correct PSF; a guessed length or angle produces confident-looking detail that was never recorded. Deconvolution also assumes one uniform blur across the frame, so a scene where only one object moved needs that object isolated first with `--roi`.

CLI: `--deblur length=15 angle=30 noise=0.01`, `--deblur-defocus radius=5 noise=0.01`

`motion_blur_psf` and `defocus_psf` build the PSFs; `apply_psf` applies one forward, which is useful for previewing what a PSF means. `wiener_deconvolution` takes an arbitrary PSF.

Because the true PSF cannot be read off an image reliably, `deblur_sweep(image, lengths, angles)` renders a labelled grid across the parameter space to judge by eye, and `focus_score(image)` ranks results by the variance of the Laplacian.

Sprint 3 — Multi-frame (not chain steps)

`frame_averaging.py` consumes a *sequence* of frames and produces one image, so it runs before the filter chain rather than inside it. On the CLI this is `--frames N`, with `--frame` selecting the start index and `--frame-step` the stride.

Function	CLI method	What it does
`average_frames(frames, weights=None)`	`mean`	Suppresses random noise; noise falls with the square root of the frame count
`median_frames(frames)`	`median`	Removes anything present in fewer than half the frames, reconstructing the background
`integrate_frames(frames, gain, auto_scale)`	`integrate`	Accumulates light from very dark footage without amplifying noise the way gain would
`sharpest_frames(frames, count)`	`sharpest`	Ranks frames by focus; the CLI averages the best-focused half

All of them assume the frames are **aligned**. Handheld or PTZ footage needs stabilising first — a moving camera turns averaging into a blur.

`frame_difference(a, b, amplify)` gives the absolute difference between two frames, for isolating what moved.

CLI: `--frames 24 --frame-method median --frame-step 5`

Remaining catalogue

Each module's own docstring carries the full reasoning; this is the summary and the caveats that matter most.

Adjust

`curves` — control-point tonal curve, interpolated with a monotonic (PCHIP) spline so the mapping can never double back and invert the tonal order the way a plain cubic can. Presets: `linear`, `brighten`, `darken`, `contrast`, `reduce_contrast`, `lift_shadows`, `film`. CLI: `--curves preset=lift_shadows` or `--curves points=0:0,128:170,255:255`.

`white_balance` — each automatic method assumes something about the scene, and fails when it does not hold: `gray_world` (the average is neutral — fails when one colour dominates), `white_patch` (the brightest pixels are white — fails on a blown highlight), `shades_of_gray` (a compromise, and the default). When the scene contains something known to be neutral, `--wb-patch X,Y,W,H` measures it instead of guessing.

`saturation` / `vibrance` — vibrance weights the boost towards muted colours so vivid ones do not clip into flat blocks. Note the falloff is *proportional*: a mid-saturation colour can still gain more raw saturation than a nearly-neutral one.

`color_balance` — shifts shadows, midtones and highlights independently, with overlapping Gaussian weights so ranges blend rather than band. Useful when two light sources cast differently by brightness. `preserve_luminosity` keeps overall brightness fixed so only colour moves.

`invert` — `invert_luminance` flips brightness while keeping hue, which sometimes makes faint dark-on-dark detail readable where raising brightness only washes it out.

Enhance

`nl_means` — averages patches that *look alike* wherever they sit, so repeating texture reinforces rather than smooths away. The slowest denoiser here; cost grows with the square of `search_window`. Set `h` from measured noise via `estimate_h`, or use `--nl-means-auto`. `nl_means_denoise_frames` uses neighbouring frames as extra evidence without the smearing plain frame averaging causes on moving objects.

`super_resolution` — the distinction here matters more than anywhere else in the toolkit. `upscale` **interpolates and adds no information**; a plate unreadable at native resolution stays unreadable enlarged. `super_resolve` genuinely recovers detail, because sub-pixel motion between frames samples the scene on different grids. It needs real sub-pixel motion — `super_resolve_report` tells you whether a sequence has any.

Phase correlation drives the alignment, and it needs broadband detail. A strongly periodic scene (tiles, brickwork, a fence) produces several correlation peaks of similar height and the measured offset can be meaningless rather than merely imprecise.

`detail_enhancement` — `local_contrast` is a large-radius unsharp mask, which is what most "clarity" sliders are. `enhance_detail` is edge-preserving, so it lifts texture without halos. `multiscale_detail` boosts frequency bands independently.

Correct

`perspective` — four-point rectification. Pass the surface's known real-world ratio (`KNOWN_RATIOS`: `a4_portrait`, `a4_landscape`, `us_letter`, `credit_card`, `plate_eu`, `plate_us`, `square`) because estimating it from a perspective

view is unreliable. `find_document_corners` detects a rectangular surface automatically; it returns None on a cluttered scene, which is the expected outcome rather than an error.

`fisheye_correction` — `barrel` uses the polynomial radial model (hand-tuned, fine for moderate wide-angle); `fisheye` uses the equidistant model for true dome cameras. Both *estimate* the distortion. When the camera is available, calibrate instead.

`aspect_ratio` — SD video does not use square pixels; displayed uncorrected, everything is stretched and any measurement is wrong in one axis. `PIXEL_ASPECT_RATIOS` covers PAL, NTSC, HDV and anamorphic. `fit_to_aspect` mode `pad` is the only one that alters neither geometry nor content.

`undistort` — the defensible route: derive the camera's actual intrinsics from chessboard photographs, then invert exactly that. `CameraCalibration.is_reliable` checks the reprojection error is under one pixel. A calibration is specific to one camera at one zoom and focus; applying another camera's is worse than applying none, because the result looks plausible while being geometrically wrong.

Analyze

`compression_analysis` — `blockiness_score` compares intensity steps across the 8-pixel JPEG grid against steps elsewhere. `estimate_jpeg_quality` reads the quantisation tables directly from a JPEG, which is exact rather than inferred — but only while the file is still a JPEG.

The measure assumes photographic content. An image dominated by hard synthetic edges that miss the block grid inflates the interior term and can read as no blocking whatever its history. Strong blocking means heavy compression, nothing more; re-saving normalises the grid and erases any local difference.

Special

`color_deconvolution` — separates overlapping colorants (ink over print, stamp over signature) by solving in optical density, where absorption is additive. At most three colorants, since three channels give three equations, and colorants with near-parallel colour vectors separate poorly. `estimate_stain_vector` measures a vector from a patch of one colorant alone, which is the reliable way to build a separation for an unknown ink.

`component_separation` — detail invisible in a colour composite is often plain in one component. Colour spaces (`rgb`, `hsv`, `hls`, `lab`, `luv`, `ycrcb`, `yuv`, `xyz`), frequency split (base versus detail), and bit planes. Structure in the low bit planes is notable: natural sensor noise has none, so a pattern there suggests hidden data or a pasted region.

`redaction` — the one operation that must not be undoable, and the obvious methods fail. **Blurring is reversible** — it is a known convolution, and this toolkit's own Wiener deconvolution will undo it. **Pixelation is reversible for short known-alphabet text** — rendering every candidate plate and matching block means is a documented, cheap attack. Only `fill` and `noise` discard the original pixels; `fill` is the default and the only method for a document intended for release. `verify_redaction` correlates each region against the original and reports whether the content actually went.

`annotate` — arrows, shapes, text, and calibrated measurement. `Scale` converts pixels to units; `measure_distance` (1D), `measure_area` (2D, shoelace formula), `draw_measurement` and `draw_scale_bar` present them.

A scale is valid only for the plane it was measured in. A ruler on the ground calibrates distances on the ground and says nothing about a sign three metres behind it, which is further from the camera and smaller per pixel. Correct perspective first. Annotations are drawn on a copy — keep the unannotated original, since a marked-up image is a figure, not evidence.

`measure_3d` — height out of the ground plane, which the scale above cannot reach. Given the ground plane's horizon, the vanishing point of scene verticals, and one reference object of known height standing on that ground, the height of anything else standing on the same ground follows from a cross-ratio — a quantity projection

preserves. The method is Criminisi, Reid and Zisserman, *Single View Metrology*, IJCV 40(2), 2000.

Parameter	Default	Meaning
`base`, `top`	required	Target's ground contact and highest point
`reference_base`, `reference_top`	required	Same two points on the known object
`horizon`	required	A `y` row for a level camera, `x1,y1,x2,y2`, or `a,b,c`
`reference_height`	`1800.0`	True height of the reference, in `unit_name`
`vertical_point`	`None`	Vertical vanishing point; omit if verticals are parallel
`unit_name`	`'mm'`	Unit label
`show_horizon`	`True`	Draw the horizon the estimate rests on

`measure_height` returns the number without drawing, along with `uncertainty_per_pixel` — how far the answer moves for a one-pixel error in the clicked points. Read it before quoting a height; it is the floor on the error, not the whole of it. `vanishing_point` solves for a vanishing point from two or more scene-parallel lines by least squares, and `horizon_from_vanishing_points` builds the horizon from two of them.

The estimate is only as good as its assumptions, and each one fails quietly:

- **Both objects must stand on the same ground plane.** Someone on a kerb, a step or a slope is being measured against a plane they are not on.
- **Correct lens distortion first.** Curved straight lines put the vanishing geometry wrong before any arithmetic happens.
- **The base is the ground contact.** A gap under a heel, or feet hidden behind a car, biases the result directly.
- **Accuracy collapses near the horizon**, where one pixel spans a fast-growing real distance — which is what `uncertainty_per_pixel` is reporting.
- **Omitting `vertical_point` assumes the camera has no tilt.** Against a synthetic camera 2.5 m up with a 1.8 m reference, that assumption over-read by about 16 mm at 5° of pitch and 33 mm at 18°. Most CCTV is tilted, so supply the vertical vanishing point.

ROI helpers (not chain steps)

`analyze_roi(image, roi)` returns per-channel mean, std, min and max plus pixel count. Exposed on the CLI as `--analyze-roi X,Y,W,H`, which runs against the **processed** image.

`apply_to_roi(image, roi, filter_fn, **kwargs)` applies any filter to a region only, leaving the rest of the frame untouched.

`get_centered_roi(shape, w, h)` and `roi_from_ratio(shape, x, y, w, h)` build regions without hardcoding pixel coordinates.